The Most Efficient Protocol for PAKE:
What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, xujiay@oregonstate.edu

1 Introduction

2 Preliminaries

3 Security Analysis of the "Hash Everything" Version

3.1 The Simulator

3.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Simulating honest P-to-P' message. When P's session begins, $\mathcal{S}$ must simulate its message (s, T) to P'. This can be easily done: just sample (s, T) at random. However, every time $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), $\mathcal{S}$ needs to generate a KEM key pair (pk^*, sk^*) for later use (see the "Extracting password guess from malicious P'" paragraph below) and make sure that (pk^*, sk^*) is consistent with the RO queries. This can be done as follows: $\mathcal{S}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*,$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. This also allows $\mathcal{S}$ to store the association between pw^* and (pk^*, sk^*). (Note that (pk^*, sk^*) needs to be freshly generated every time $\mathcal{A}$ queries $H_2(\star, T)$; otherwise R^* never changes and $\mathcal{A}$ can easily find collisions in H_1.)

Extracting password guess from malicious P. When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P', if it is different from the honest message (s, T), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding

TestPwd command to $\mathcal{F}_{\text{PAKE}}$. There are two types of attack, which need to be addressed separately:

1. *"Normal" online attack:* $\mathcal{A}$ executes P's algorithm on a password guess pw^*. That is, $\mathcal{A}$ samples public key pk^* and randomness r^*, computes

$$R^* := H_1(\mathsf{pw}^*, r^*), \qquad T^* := pk^* \cdot R^*,$$
$$t^* := H_2(\mathsf{pw}^*, T^*), \qquad s^* := t^* \oplus r^*,$$

   and sends (s^*, T^*) to P'.
   It takes a while to realize how to extract pw^* from (s^*, T^*). First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values (and multiple t^* values); $\mathcal{S}$ must decide which one is the "actual" password guess. This can be done via the following. For each such (pw^*, t^*) pair, $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\mathsf{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\mathsf{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the "actual" password guess pw^*. (This special $H_1(\mathsf{pw}^*, r^*)$ query is called the "anchor query" in [MRR20, JRX25].)

2. *Related key attack:* This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P. $\mathcal{A}$ skips sampling pk^* or r^*; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$t^* := H_2(\mathsf{pw}^*, T), \qquad (t^*)' := H_2(\mathsf{pw}^*, T^*),$$
$$\delta := t^* \oplus (t')^*, \qquad s^* := s \oplus \delta,$$

   and sends (s^*, T^*) to P'.
   Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*), such that

$$s \oplus s^* = H_2(\mathsf{pw}, T) \oplus H_2(\mathsf{pw}, T^*),$$

   which allows $\mathcal{A}$ to pick T^* and "reverse engineer" the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^*/H_1(\mathsf{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^*/T)$.
   To extract pw^* from (s^*, T^*), $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which ones satisfy

$$H_2(\mathsf{pw}^*, T) \oplus H_2(\mathsf{pw}^*, T^*) = s \oplus s^*.$$

   With overwhelming probability, at most one pw^* will satisfy this equation.

Extracting password guess from malicious P'. When the adversary $\mathcal{A}$ sends a message (c^*, τ^*) to P, if $\mathcal{A}$ is not passive (i.e., it is not the case that $(s^*, T^*) = (s, T)$ *and* $(c^*, \tau^*) = (c, \tau)$), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding TestPwd command to $\mathcal{F}_{\text{PAKE}}$.

$\mathcal{S}$ should find the H_{tag} query whose result is τ^*, which includes $(\mathsf{pw}^*, pk^*, K^*)$. However, pw^* constitutes a password guess only if it is consistent with pk^* and K^*, i.e., $K^* = \mathsf{Decap}(sk^*, c^*)$ for sk^* corresponding to pk^*. $\mathcal{S}$ can check this condition because it has stored the correspondence between pw^* and (pk^*, sk^*) (see the "Simulating honest P-to-P' message" paragraph above), and can use sk^* to check if the equation holds. (This extraction strategy does not work anymore if *any* of pw^*, pk^*, c^* is not included in H_{tag}. In later sections we will explain how to simulate other versions of the protocol.)

3.1.2 Formal Description of the Simulator

3.2 Hybrid Argument

We now prove that for any PPT environment $\mathcal{Z}$, the simulator $\mathcal{S}$ in Section 3.1.2 generates an ideal-world view indistinguishable from $\mathcal{Z}$'s real-world view. The sequence of games start from the real world (Game 0) and ends at the ideal world.

Game 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; the flow of events is:

- P sends (s, T) to P';
- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who sends (s^*, T^*) (which may or may not be equal to (s, T)) to P';
- P' outputs session key SK' and sends (c, τ') to P;
- It is again intercepted by $\mathcal{A}$, who sends (c^*, τ^*) to P;
- P computes τ. If $\tau^* = \tau$ then P outputs SK, otherwise P outputs $\perp$.

(Note that there are three τ values: τ' computed and sent by P', τ^* sent by $\mathcal{A}$, and τ computed by P used in P's check.)

$\mathcal{A}$ is passive, except maybe modifying the tag.

Game 1: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do the following:

- If $\tau^* = \tau'$ (i.e., $\mathcal{A}$ is passive), set $SK := SK'$.
- If $\tau^* \neq \tau'$, set $SK := \perp$.

By correctness of the protocol (see Section 2), in Game 0 $\tau = \tau'$ except with probability $\mathbf{Adv}^{\text{correctness}}$. This means that except with probability $\mathbf{Adv}^{\text{correctness}}$, P's check $\tau^* \stackrel{?}{=} \tau$ in Game 0 is equivalent to P's check $\tau^* \stackrel{?}{=} \tau'$ in Game 1. If the check passes then both games set $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c, K)$, otherwise both games set $SK := \perp$. We have

$$\mathbf{Dist}^{0,1}_{\mathcal{Z}} = \mathbf{Adv}^{\text{correctness}}.$$

This game corresponds to games G_3–G_5 in [ABJ25]. We believe having a single game that handles both the $\tau^* = \tau'$ case and the $\tau^* \neq \tau'$ case is much cleaner.

P and P''s passwords match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 2: In the case where $\tau^* \neq \tau'$, set $SK := \perp$ if either of the following happens:

- $\mathcal{A}$ never makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, or
- $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ queries, both resulting in τ^*.

In the first event, Game 1 and Game 2 differ if $\mathcal{A}$ never makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, yet P's check $\tau^* \stackrel{?}{=} H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ passes. Clearly,

- $\mathcal{A}$ does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ (otherwise the result is τ^*), and
- P' does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ (otherwise $\tau' = H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K) = \tau^*$).

So $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is independent of $\mathcal{A}$'s view, and the probability that $\mathcal{A}$ sends $\tau^* = H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is $1/2^n$. The second event is collision in H_{tag}, whose probability is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)/2^{n+1}$. Overall,

$$\mathbf{Dist}^{1,2}_{\mathcal{Z}} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1) + 2}{2^{n+1}}.$$

This game corresponds to games G_6 and G_7 in [ABJ25], except that the H_{tag} collision case was ruled out in game G_2 in [ABJ25] (as aborting condition [A1]).

Game 3: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as before.)

Let Query_1 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(\mathsf{pw}, pk, c, K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K')$. Clearly Game 2 and Game 3 are identical unless Query_1 happens. We upper-bound $\Pr[\mathsf{Query}_1]$ via a reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property of KEM:

Reduction $\mathcal{R}_{\text{OW-1PCA1}}$: // Boxed text indicates the single PC oracle query; same for reductions below

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_1.
On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW-PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}(\mathsf{pw}, pk, c, (K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_1 happens, the only difference between Game 2/3 and $\mathcal{R}_{\text{OW-1PCA1}}$'s simulation (until Query_1 happens) is that $\mathcal{R}_{\text{OW-1PCA1}}$ uses its input pk on the P side, and then uses its input $c \leftarrow \mathsf{Encap}(pk)$ on the P' side. Since we assume $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, in Game 2/3 P''s algorithm will recover $pk' = pk$, so there c is also computed as $\mathsf{Encap}(pk)$ and thus $\mathcal{R}_{\text{OW-1PCA1}}$ simulates Game 2/3 perfectly. Furthermore, if $\mathcal{R}_{\text{OW-1PCA1}}$'s guess is correct then it wins the OW-1PCA game. We have

$$\mathbf{Dist}^{2,3}_{\mathcal{Z}} \leq \Pr[\mathsf{Query}_1] \leq (q_{\mathsf{key}} + q_{\mathsf{tag}})\mathbf{Adv}^{\text{OW-1PCA}}_{\mathcal{R}_{\text{OW-1PCA1}}}.$$

This game corresponds to game G_8 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to OW-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than OW-1PCA. In this case, step 4 of the reduction above can query $\mathsf{PC}(sk, c, (K')^*)$ $q_{\mathsf{key}} + q_{\mathsf{tag}}$ times to check which of $\mathcal{A}$'s query (if any) triggers Query_1, hence avoiding the $q_{\mathsf{key}} + q_{\mathsf{tag}}$ loss in its advantage.*

2. *The above also means that the plaintext-checking oracle in their OW-PCA game has to allow for $\mathsf{PC}(sk, c, \star)$ queries, whereas our reduction to OW-1PCA never makes a query in this form and thus the plaintext-checking oracle in our OW-1PCA game can disallow such queries.*

Game 4: Again in the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, generate P''s KEM public key $(pk', \star) \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$. (Note that K' is not used anywhere since Game 3 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.)

The difference between Game 3 and Game 4 is that in Game 3 both P and P' use the same pk, whereas in Game 4 P uses pk and P' uses an independent pk'. We construct a reduction $\mathcal{R}_{\text{ANO-1PCA}}$ to the ANO-1PCA property of KEM. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\text{ANO-1PCA}}$:

On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO-PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

$\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk on the P side and c on the P' side. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk)$, i.e., both P and P' use the same pk, so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 3; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., P' uses an independently generated pk', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 4. We have

$$\mathbf{Dist}^{3,4}_{\mathcal{Z}} \leq \mathbf{Adv}^{\text{ANO-1PCA}}_{\mathcal{R}_{\text{ANO-1PCA}}}.$$

This game corresponds to game G_9 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to ANO-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than ANO-1PCA. Unlike in Game 3, here reducing to this stronger assumption does not provide any advantage.*

2. *The assumption [ABJ25] reduces to is even further stronger in that their KEM adversary's inputs are (pk, pk', c), whereas we do not give pk' to the KEM adversary. Note that the reduction does not need pk' at all (it only needs to send c which is an encapsulation of either pk or pk'), so the assumption in [ABJ25] is unnecessarily strong.*

P and P''s passwords do not match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 5: When either $\mathcal{A}$ or an honest party makes a fresh query $H_2(\mathsf{pw}^*, T)$ (let the result be t^*)[1], generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$ and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

((s, T) are the honest P-to-P' message.) If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$, and if the $H_2(\mathsf{pw}^*, T)$ query is made by $\mathcal{A}$, also store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$. (This in particular means that in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, the generation of pk', r', R' is replaced by the method described above. Note that the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ has already been addressed in Game 3 and Game 4, and P' does not need to make an H_2 query there.)

We first upper-bound the aborting probability. For every fresh $H_2(\mathsf{pw}^*, T)$ query, an independent $t^* \leftarrow \{0,1\}^{3n}$ value is sampled, which results in an independent $r^* \leftarrow \{0,1\}^{3n}$ (regardless of the distribution of s). As such, there are up to $q_2 + 2$ independent and uniformly random r^* values. The probability that one of $\mathcal{A}$'s q_1 $H_1(\mathsf{pw}^*, r^*)$ query inputs happens to be any of them is upper-bounded by $q_1(q_2 + 2)/2^{3n}$.

Now suppose the aborting condition does not happen. The difference between Game 4 and Game 5 is that Game 4 samples $H_1(\mathsf{pw}^*, r^*)$ uniformly at random, whereas Game 5 defines it as T/pk^*. Note that pk^* is not used anywhere else in the game, so the distinguishing advantage between Game 4 and Game 5 can be reduced to the PR-PK property of KEM. We only sketch the reduction $\mathcal{R}_{\text{PR-PK}}$ here: $\mathcal{R}_{\text{PR-PK}}$, on input pk^* (which is either an honestly generated public key or a uniformly random value), samples $i \leftarrow [q_2 + 2]$, and answers the (either $\mathcal{A}$'s or an honest parties') first $i - 1$ H_2 queries as in Game 4 and the last $q_2 - i + 2$ H_2 queries as in Game 5; for the i-th H_2 query, $\mathcal{R}_{\text{PR-PK}}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. (Note that the $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record is not actually used in Game 5, so $\mathcal{R}_{\text{PR-PK}}$ does not need to store it.) $\mathcal{R}_{\text{PR-PK}}$ simulates other parts of Game 4/5 honestly. Finally, $\mathcal{R}_{\text{PR-PK}}$ copies $\mathcal{Z}$'s output bit.

[1] When P makes such a query, it is in the form of $H_2(\mathsf{pw}, T)$ and the result is t; when P' makes such a query, it is in the form of $H_2(\mathsf{pw}', T)$ and the result is t'. Of course P' makes this query only if $T^* = T$.

A standard hybrid argument shows that (assuming the aborting condition does not happen) $\mathcal{Z}$'s distinguishing advantage between Game 4 and Game 5 is upper-bounded by $q_2 \cdot \mathbf{Adv}^{\text{PR-PK}}_{\mathcal{R}_{\text{PR-PK}}}$.[2] Overall, we have

$$\mathbf{Dist}^{4,5}_{\mathcal{Z}} \leq (q_2+2)\mathbf{Adv}^{\text{PR-PK}}_{\mathcal{R}_{\text{PR-PK}}} + \frac{q_1(q_2+2)}{2^{3n}}.$$

This game corresponds to game G_{10} in [ABJ25], except that the aborting case was ruled out in game G_2 in [ABJ25] (as aborting condition [A2b1]).

Game 6: In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$.

Let Query_2 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(\mathsf{pw}', pk', c, K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}', pk', c, K')$. Clearly Game 5 and Game 6 are identical unless Query_2 happens. We upper-bound $\Pr[\mathsf{Query}_2]$ via a reduction $\mathcal{R}_{\text{OW1}}$ to the OW property of KEM. This is somewhat similar to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3, and we highlight the differences in blue:

Reduction $\mathcal{R}_{\text{OW1}}$:

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_2, and $j \leftarrow [q_2 + 1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.
2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\text{OW1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\text{OW1}}$'s input pk' instead of generating pk^* freshly.
3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise
   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.
   (b) Sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW1}}$'s inputs.)
4. On (c^*, τ^*) from $\mathcal{A}$,
   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

[2] Note that here we cannot make q_2 separate reductions to the PR-PK property, and must reduce to PR-PK "in one blow". Furthermore, the calculation of the reduction's advantage is more complicated than the normal case where the reduction wins as long as its guess is correct. This is similar to, say, why composing a PRG polynomial times yields another PRG; see, e.g., [BS23, Section 3.4.3] for details.

(b) Compute $K := \mathsf{Decap}(sk, c)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
If $K = K^*$, output $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K)$ to $\mathcal{Z}$.
If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.

5. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}(\mathsf{pw}', pk', c, (K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}', pk', c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_2 happens and $\mathcal{R}_{\mathrm{OW1}}$'s guess of index j is correct, the only difference between Game 5/6 and $\mathcal{R}_{\mathrm{OW1}}$'s simulation (until Query_2 happens) is that $\mathcal{R}_{\mathrm{OW1}}$ uses its input pk' to program $H_1(\mathsf{pw}', r')$, and then uses its input $c \leftarrow \mathsf{Encap}(pk')$ as the P'-to-P message. Since c is also computed as $\mathsf{Encap}(pk')$ in Game 5/6, $\mathcal{R}_{\mathrm{OW1}}$ simulates Game 5/6 perfectly. Furthermore, if $\mathcal{R}_{\mathrm{OW1}}$'s guess of index i is correct then it wins the OW game. We have

$$\mathbf{Dist}^{5,6}_{\mathcal{Z}} \leq \Pr[\mathsf{Query}_2] \leq (q_2 + 1)(q_{\mathsf{key}} + q_{\mathsf{tag}})\mathbf{Adv}^{\mathrm{OW}}_{\mathcal{R}_{\mathrm{OW1}}}.$$

Remark. It is very subtle why the reduction needs to guess the index j and lose a factor of $q_2 + 1$ (and we applaud [ABJ25] for getting it right). The reason is as follows. Consider the following environment $\mathcal{Z}$:

1. Initiate P's session on password pw and receive (s, T).
2. Instruct $\mathcal{A}$ to make a large number of $H_2(\mathsf{pw}^*, T)$ queries (let the result be t^*), one of which is $H_2(\mathsf{pw}', T)$ for a special value pw'.
3. For each of the H_2 queries in step 2,
$$\text{compute } r^* := s \oplus t^*, \quad \text{query } H_1(\mathsf{pw}^*, r^*).$$
4. Initiate P''s session on password pw' and instruct $\mathcal{A}$ to forward (s, T) to P'.

To simulate Game 5/6 correctly, $\mathcal{R}_{\mathrm{OW1}}$ must use its input pk' to program $H_1(\mathsf{pw}', r') := T/pk'$ in step 3. The problem is that $\mathcal{R}_{\mathrm{OW1}}$ *does not know* pw' *until step 4, so it has no idea which* $H_1(\mathsf{pw}', r')$ *query in step 3 needs to be programmed using* pk'! The only way to get around is to make a guess over all H_2 queries, which incurs a $q_2 + 1$ loss.[3] (If we require $\mathcal{Z}$ to always initiate P and P''s sessions simultaneously, this loss can be avoided since in step 3 $\mathcal{R}_{\mathrm{OW1}}$ must have already learned pw'.)

This game corresponds to game G_{11} *in [ABJ25], except that they reduce to OW-PCA and avoids the* $q_{\mathsf{key}} + q_{\mathsf{tag}}$ *loss (cf. the comment after Game 3).*

[3] It is $q_2 + 1$ rather than $q_2 + 2$, since P's $H_2(\mathsf{pw}, T)$ query is guaranteed to be different from $H_2(\mathsf{pw}', T)$ (otherwise $\mathcal{R}_{\mathrm{OW1}}$ would abort in step 3) and thus can be excluded from the guess.

Game 7: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, recall Game 6 does the following on the P' side: Generate $(pk', \star) \leftarrow \mathsf{Gen}$, query $t' := H_2(\mathsf{pw}', T)$, and compute

$$r' := s \oplus t', \quad R' := T/pk'.$$

((s, T) are the honest P-to-P' message.) If $H_1(\mathsf{pw}', r')$ has already been defined and it is not R', abort. Otherwise program $H_1(\mathsf{pw}', r') := R'$, compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$, and sample $SK', \tau' \leftarrow \{0,1\}^n$. Finally, output SK' and send (c, τ') to P. (Note that K' is not used anywhere since Game 6 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.) In this game, we independently generate $(pk'', \star) \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk'')$ (the underlined expression).

The difference between Game 6 and Game 7 is that in Game 6 c is generated by

$$pk' = \frac{T}{H_1(\mathsf{pw}', s \oplus H_2(\mathsf{pw}', T))}$$

which $\mathcal{A}$ can compute, whereas in Game 7 c is generated by an independent pk''. We construct a reduction $\mathcal{R}_{\mathrm{ANO1}}$ to the ANO property of KEM. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\mathrm{OW1}}$ to the OW property in Game 6 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\mathrm{ANO1}}$:

Sample $j \leftarrow [q_2 + 1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.
2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\mathrm{ANO1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\mathrm{ANO1}}$'s input pk' instead of generating pk^* freshly.
3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise
   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.
   (b) Sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\mathrm{ANO1}}$'s inputs.)
4. On (c^*, τ^*) from $\mathcal{A}$,
   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

(b) Compute $K := \mathsf{Decap}(sk, c)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
If $K = K^*$, output $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K)$ to $\mathcal{Z}$.
If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.

5. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

Assume $\mathcal{R}_{\text{ANO1}}$'s guess of index j is correct. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., the computation of R' and c use the same pk', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 6; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk'')$, i.e., the computation of c uses an independently generated pk'', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 7. We have

$$\mathbf{Dist}^{6,7}_{\mathcal{Z}} \leq (q_2 + 1)\mathbf{Adv}^{\text{ANO-1PCA}}_{\mathcal{R}_{\text{ANO-1PCA}}}.$$

Game 8: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, we outright skip the computation of t', r', R' and the programming of H_1 (the brown text in Game 6). (Since pk' is not generated anymore, we can rename pk'' — the KEM public key from which c is computed — to pk'.)

Note that the aborting condition, which was originally introduced in Game 5, is removed (for the specific $H_2(\mathsf{pw}', T)$ query by P'). By an analysis similar to that in Game 5, the aborting probability in Game 7 is upper-bounded by $q_1/2^{3n}$. If the aborting condition does not happen, the brown text is independent of the rest of the game, so removing it does not affect the distribution of $\mathcal{Z}$'s output. We have

$$\mathbf{Dist}^{6,7}_{\mathcal{Z}} \leq \frac{q_1}{2^{3n}}.$$

The two games above combined correspond to game G_{12} in [ABJ25]. [ABJ25] does not have the intermediate Game 7 in our proof, and outright removes the brown text from what's our Game 6. We believe it is clearer to include our Game 7, where H_2 is still queried and H_1 is still programmed: in this way it is easier to see why the reduction to ANO simulates the games correctly. (Of course, it is possible that an experienced cryptographer can see through the two game hops in one shot, but the author of this paper admits that he is not that experienced and has to work out the game hops and reductions the "dumb" way.) Also, [ABJ25] reduces to ANO-PCA, although it also says that the plaintext-checking oracle is not needed.

References

[ABJ25] Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. Cryptology ePrint Archive, Report 2025/231, 2025.

[BS23] Dan Boneh and Victor Shoup. A graduate course in applied cryptography (version 0.6), 2023. https://toc.cryptobook.us/book.pdf.

[JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.

[MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.